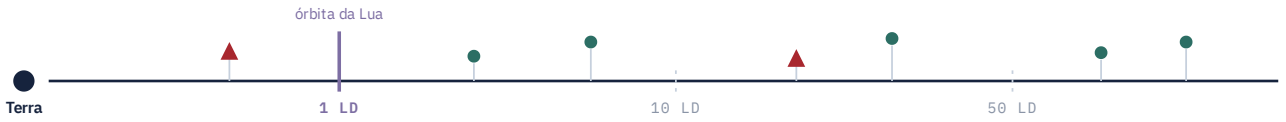
Serviços em Nuvem · SN-2026 · Grupo 01

Pipeline de dados com a API NASA NeoWs

Guia completo para replicar um pipeline automatizado que coleta aproximações de asteroides à Terra, armazena em PostgreSQL no Supabase e publica um painel web no GitHub Pages.

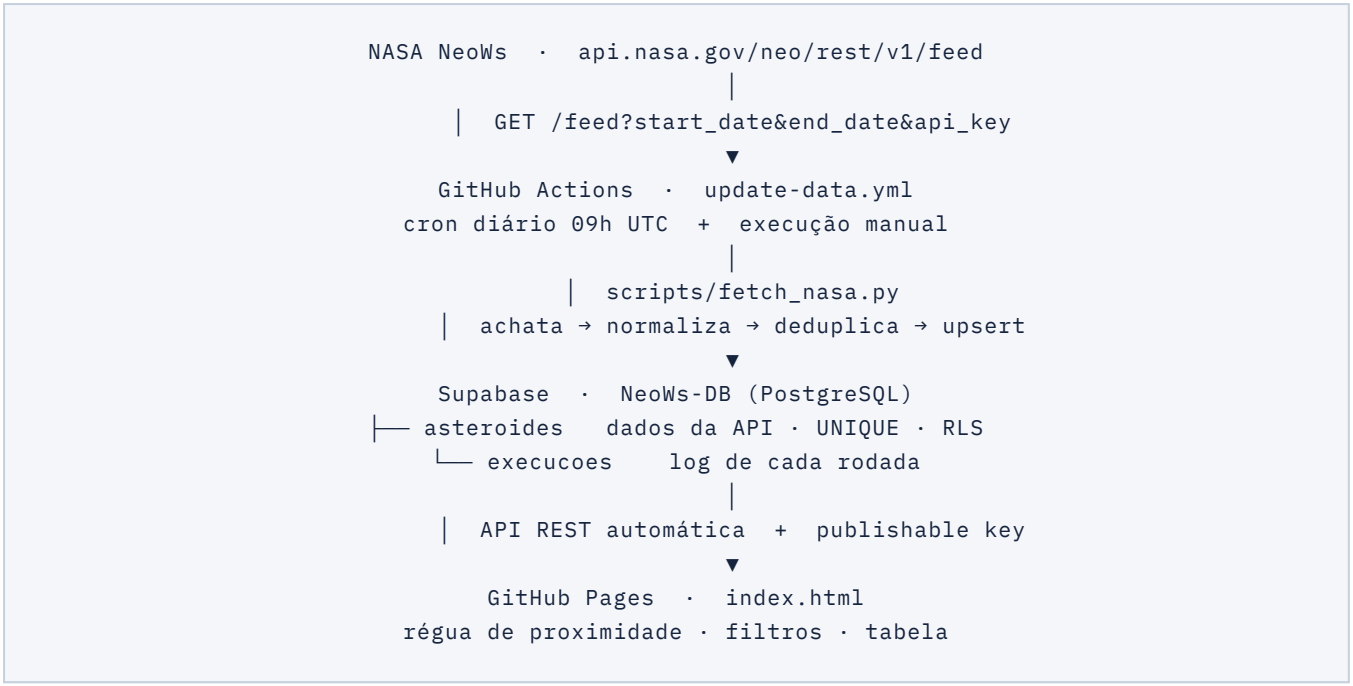


Integrantes	**João Vitor Cosmo · Maria Laura Schossler**
API	`api.nasa.gov/neo/rest/v1/feed`
Organização	`SN-2026-GRUPO-01-NASA`
Repositório	`NeoWs`
Projeto Supabase	`NeoWs-DB`
Painel	`sn-2026-grupo-01-nasa.github.io/NeoWs/`

1 O que será construído

Ao final deste guia você terá um pipeline rodando sozinho todo dia, gravando dados reais num banco PostgreSQL e alimentando um painel público.

Três serviços conversam entre si. O **GitHub Actions** executa um script Python em horário agendado; esse script consulta a **API da NASA** e grava os resultados no **Supabase**; o **GitHub Pages** serve uma página que lê esses dados pela API REST do próprio Supabase.



Por que essa separação

O script nunca roda no navegador de ninguém. Ele roda no servidor do GitHub, onde a chave de escrita do banco fica protegida como *Secret*. O navegador só recebe uma chave de leitura, que não consegue alterar nada — a segurança vem da combinação entre RLS, políticas e GRANTs no banco, explicada na seção 5.

Pré-requisitos

Item	Observação
Conta no GitHub	Gratuita. Será necessário criar uma organização.
Conta no Supabase	Gratuita. O plano free limita projetos ativos simultâneos.
Chave da NASA	Gratuita e instantânea em `api.nasa.gov`.
Navegador	Todo o processo pode ser feito pela interface web, sem terminal.

2 A API NeoWs

Antes de escrever qualquer código, vale entender três particularidades do endpoint. Ignorar qualquer uma delas custa retrabalho.

A **NeoWs** (Near Earth Object Web Service) é mantida pelo time de Asteroides do NASA JPL e expõe informações sobre objetos que se aproximam da Terra. O endpoint usado aqui é o `/feed`, que devolve as aproximações dentro de um intervalo de datas.

```
https://api.nasa.gov/neo/rest/v1/feed?start_date=2026-11-13&end_date=2026-11-19&api_key=SUA_CHAVE
```

Particularidade 1 — janela máxima de sete dias

A API recusa intervalos maiores que sete dias entre `start_date` e `end_date`. O script limita a janela automaticamente, mas é bom saber: para carregar um mês inteiro são necessárias cinco execuções com datas diferentes.

Particularidade 2 — a resposta não é uma lista

O campo `near_earth_objects` não devolve um array. É um objeto cujas *chaves são datas*, e cada data contém um array de asteroides:

```
{
  "element_count": 31,
  "near_earth_objects": {
    "2026-09-28": [ { ...asteroide... }, { ...asteroide... } ],
    "2026-09-29": [ { ...asteroide... } ]
  }
}
```

Por isso o script precisa **achatar** a estrutura: percorrer as datas, e dentro de cada uma percorrer os asteroides, gerando uma linha por aproximação.

Particularidade 3 — um asteroide, várias aproximações

Cada objeto traz um array `close_approach_data` que pode conter passagens de outras datas. No modo *feed* a API normalmente já filtra, mas o script confere a data de cada entrada antes de gravar — é o que garante que uma aproximação de 2027 não entre no registro de hoje.

Campos aproveitados

Campo na API	Coluna no banco
`neo_reference_id`	identificador do objeto — parte da chave natural
`name`	nome do asteroide
`absolute_magnitude_h`	magnitude absoluta
`estimated_diameter.meters`	diâmetro mínimo, máximo e média calculada
`is_potentially_hazardous_asteroid`	classificação de risco
`close_approach_date`	data da passagem — parte da chave natural
`relative_velocity`	velocidade em km/s e km/h
`miss_distance`	distância em km, em distâncias lunares e em UA

Distância lunar (LD). É a unidade que a área usa no dia a dia: 1 LD equivale à distância média entre a Terra e a Lua, cerca de 384.400 km. Um asteroide que passa a 0,8 LD passou mais perto do que a Lua. O painel usa essa unidade como referência visual principal.

3 Obter a chave da NASA

Primeiro passo prático, porque a chave é dependência de tudo que vem depois.

1. Acesse **api.nasa.gov** e localize o formulário *Generate API Key*.
2. Preencha nome, sobrenome e e-mail. Não é necessário criar senha nem confirmar cadastro.
3. Clique em **Signup**. A chave aparece na tela imediatamente e também chega por e-mail. Guarde em local seguro.

> **Não use a DEMO_KEY.** A NASA oferece uma chave de demonstração para testes rápidos, mas o limite é de poucas requisições por hora e compartilhado entre todos os usuários do mundo. Em produção o pipeline falharia com HTTP 429.

> **Chave é credencial.** Ela não vai em commit, print, mensagem de chat nem no `index.html`. O único lugar dela é o cofre de Secrets do GitHub, configurado na seção 6. Se for exposta por acidente, gere outra.

4 Organização e repositório no GitHub

A nomenclatura é padronizada. Nomes de repositório não aceitam espaços nem barras, então a API com barra vira nome com hífen.

4.1 Criar a organização

1. Acesse `github.com/organizations/plan` e escolha o plano **Free**.
2. Nome da organização, exatamente:

```
SN-2026-GRUPO-01-NASA
```

4.2 Criar o repositório

3. Dentro da organização, clique em **New repository**.
4. Nome `NeoWs`, visibilidade **Public** — obrigatório para o GitHub Pages gratuito. Não marque nenhuma opção de inicialização.

4.3 Convidar os avaliadores

5. Na organização, **People → Invite member**, e convide o usuário `GilFerques`. Convide também os demais integrantes do grupo.

4.4 Enviar os arquivos

No repositório vazio, use **Add file → Upload files** e arraste *dois arquivos soltos* e *três pastas inteiras*:

```
index.html          arquivo solto
README.md           arquivo solto
scripts/            pasta inteira — contém fetch_nasa.py
sql/                pasta inteira — contém setup.sql
docs/               pasta inteira — documentação
```

A pasta `.github` não sobe por upload. A interface web do GitHub ignora pastas cujo nome começa com ponto. O arquivo do workflow precisa ser criado manualmente: clique em **Add file → Create new file** e digite o caminho completo no campo do nome:

```
.github/workflows/update-data.yml
```

Ao digitar as barras, o GitHub cria as pastas automaticamente. Cole o conteúdo do workflow e confirme com *Commit changes*. Se o arquivo foi criado corretamente, o workflow aparece na aba **Actions**.

4.5 Estrutura final esperada

```
NeoWs/
├── index.html                    painel publicado no Pages
├── README.md
├── scripts/
│   └── fetch_nasa.py             coleta, deduplicação e upsert
├── sql/
│   └── setup.sql                 tabelas, constraint, RLS e GRANTs
```

```
├── .github/workflows/
│   └── update-data.yml             agendamento do pipeline
└── docs/
    ├── tutorial.pdf
    ├── ai-interaction.md
    ├── reflexao.md
    └── apresentacao.pdf
```

5 Banco de dados no Supabase

Duas tabelas, uma constraint que evita duplicatas e três camadas de permissão. Esta é a parte que mais rende pergunta na arguição — vale entender, não só executar.

5.1 Criar organização e projeto

1 Em `app.supabase.com`, crie uma organização com o mesmo nome usado no GitHub: `SN-2026-GRUPO-01-NASA`.

2 **New project** com nome `NeoWs-DB` e região **South America (São Paulo)**. Guarde a senha do banco.

> **Limite de projetos.** O plano gratuito restringe quantos projetos ficam ativos ao mesmo tempo. Se o aviso aparecer, pause ou exclua um projeto de atividade anterior.

5.2 Executar o schema

3 Abra **SQL Editor → New query**, cole todo o conteúdo de `sql/setup.sql` e clique em **Run**. O script é idempotente: pode ser reexecutado sem perder dados.

5.3 As duas tabelas

A tabela `asteroides` guarda uma linha por aproximação. A tabela `execucoes` registra cada rodada do pipeline — é dela que o painel lê a informação de última atualização.

Coluna em execucoes	Para quê
`registros_processados`	quantas linhas entraram no banco
`lotes`	quantos lotes foram enviados com sucesso
`erros`	quantos lotes falharam
`status`	concluido · erro_parcial · erro_critico · sem_dados

5.4 A chave natural e o upsert

Um mesmo asteroide aparece em datas diferentes, então o identificador sozinho não serve como chave. A combinação que identifica um registro de forma única é o objeto mais a data da passagem:

```
ALTER TABLE asteroides ADD CONSTRAINT asteroides_unique
    UNIQUE (neo_reference_id, data_aproximacao);
```

É essa constraint que permite o `upsert`: rodar o pipeline duas vezes no mesmo período *atualiza* as linhas em vez de duplicá-las. Sem ela, cada execução criaria cópias dos mesmos asteroides.

5.5 Segurança em três camadas

Publicar a chave de leitura no `index.html` só é seguro porque três mecanismos trabalham juntos. Entender a diferença entre eles é o que distingue uma configuração correta de uma que apenas parece correta.

Camada	O que faz
RLS	Liga o controle por linha na tabela. Sem RLS ativo, as políticas nem são avaliadas.
Policy	Define *quem* pode fazer o quê em quais linhas. Aqui existe apenas política de `SELECT` com `USING (true)` — leitura pública. Não há política de escrita.
GRANT	Define, no nível do banco, o que cada role pode executar na tabela. O role `anon` recebe só `SELECT`; o `service_role` recebe `ALL`.

A ausência de política de escrita é deliberada: apenas o `service_role` grava, e ele ignora RLS por definição — é por isso que a chave dele fica no servidor. Quem tem a chave pública consegue ler os asteroides e nada mais.

5.6 Convidar o professor

4 **Settings → Team → Invite**, e-mail `rafael.ferques@ifpr.edu.br`, permissão **Developer**.

5.7 Localizar as chaves

Em **Settings → API** estão os três valores necessários. As duas chaves são strings longas começando com `eyJ`, então confira o rótulo antes de copiar.

Valor	Destino
Project URL	Secret no GitHub *e* `index.html`
anon / publishable key	somente `index.html` — pode ser pública
service_role / secret key	somente Secret no GitHub — nunca exposta

6 Ligar GitHub e Supabase

Três Secrets no servidor, uma chave pública no painel e o Pages ativado.

6.1 Criar os Secrets

No repositório: **Settings → Secrets and variables → Actions → New repository secret**. Os nomes precisam ser exatamente estes, porque o workflow os procura por nome.

Nome do Secret	Valor
`SUPABASE_URL`	a Project URL do Supabase
`SUPABASE_SERVICE_KEY`	a service_role / secret key
`NASA_API_KEY`	a chave obtida em api.nasa.gov

> **Erro mais comum.** Copiar a anon no lugar da `service_role`. As duas se parecem, mas com a anon o pipeline falha na escrita — o role não tem permissão de `INSERT`.

6.2 Ativar o GitHub Pages

1. **Settings → Pages**. Em *Source*, escolha **Deploy from a branch**.
2. Branch `main`, pasta `/ (root)`, e clique em **Save**. O link aparece em verde no topo da página após um ou dois minutos.

6.3 Configurar o painel

Abra o `index.html` no GitHub, clique no ícone de lápis e localize as duas constantes no início do bloco `<script>`:

```
const SUPABASE_URL      = "https://XXXXXXXXXXXX.supabase.co";
const SUPABASE_ANON_KEY = "SUA_PUBLISHABLE_KEY_AQUI";
```

Substitua pelos valores reais do seu projeto e confirme com *Commit changes*. A URL aparece tanto aqui quanto no Secret — isso é esperado. A chave, não: aqui vai a **anon**.

7 O pipeline de coleta

Como o script funciona por dentro, e por que cada decisão foi tomada.

7.1 Sequência de execução

1. Lê as credenciais das variáveis de ambiente e encerra imediatamente se faltar alguma.
2. Monta a janela de datas, limitando a sete dias.
3. Consulta o endpoint `/feed`.
4. Achata a resposta agrupada por data em uma lista de aproximações.
5. Deduplica pela chave natural.
6. Envia em lotes de 500 registros com `upsert`.
7. Grava o log em `execucoes`.
8. Encerra com código de erro se algum lote falhou.

7.2 Por que deduplicar antes de enviar

O PostgreSQL não permite que um único comando `ON CONFLICT DO UPDATE` afete a mesma linha duas vezes. Se o mesmo par (objeto, data) aparecer repetido dentro de um lote, o envio inteiro falha com o erro **21000**:

```
ON CONFLICT DO UPDATE command cannot affect row a second time
```

A função `deduplicar()` percorre a lista guardando as chaves já vistas num conjunto e descarta as repetições antes do envio:

```
def deduplicar(lista: list) -> list:
    vistos = set()
    resultado = []
    for r in lista:
        chave = (r.get("neo_reference_id"), r.get("data_aproximacao"))
        if chave not in vistos:
            vistos.add(chave)
            resultado.append(r)
    return resultado
```

> **Duas proteções diferentes.** A constraint `UNIQUE` evita duplicatas *entre execuções*; a função `deduplicar()` evita duplicatas *dentro do mesmo lote*. Uma não substitui a outra.

7.3 Falhar de forma visível

Um pipeline que falha em silêncio é pior do que um que não roda: o ícone fica verde e ninguém percebe que o banco parou de receber dados. Por isso o script encerra com `sys.exit(1)` sempre que um lote falha, o que marca a execução como vermelha no Actions. O status também fica registrado em `execucoes`:

Status	Quando ocorre
`concluido`	todos os lotes enviados sem erro
`erro_parcial`	parte dos lotes falhou — encerra com exit 1
`erro_critico`	nenhum lote foi enviado
`sem_dados`	a API não retornou aproximações na janela

7.4 Boas práticas no workflow

Propriedade	Motivo
`permissions: contents: read`	permissão mínima. O workflow só precisa ler o repositório para o checkout — não escreve nada nele.
`concurrency`	impede duas execuções simultâneas gravando no mesmo banco.
`timeout-minutes: 10`	encerra jobs travados, poupando minutos do plano gratuito.
`workflow_dispatch`	permite execução manual com data e janela escolhidas, além do agendamento.

7.5 Executar

Aba **Actions → Pipeline NeoWs → Supabase → Run workflow**. Os campos são opcionais: em branco, o pipeline coleta os próximos sete dias a partir de hoje.

8 O painel

Uma página estática que lê o banco pela API REST, sem servidor intermediário.

8.1 Como os dados chegam

O Supabase expõe automaticamente uma API REST sobre as tabelas. O painel monta a URL com os filtros e envia a chave pública nos cabeçalhos:

```
GET /rest/v1/asteroides?select=*&order=distancia_lunar.asc&limit=500
Headers:  apikey: <anon>   Authorization: Bearer <anon>
```

8.2 A régua de proximidade

O elemento central do painel é uma régua horizontal em distâncias lunares, com a Terra na origem e a órbita da Lua marcada em 1 LD. Cada aproximação vira um marcador posicionado pela distância.

A escala é **logarítmica** porque as distâncias variam de frações de LD a centenas — numa escala linear, tudo que passa perto ficaria empilhado sobre a Terra.

Objetos potencialmente perigosos usam triângulo vermelho; os demais, círculo verde. A forma diferencia além da cor, para a leitura não depender só dela.

8.3 Filtros

- Período — data inicial e final da aproximação
- Busca por nome, com correspondência parcial
- Recorte apenas de objetos potencialmente perigosos
- Ordenação por proximidade, data, velocidade ou tamanho

8.4 Proteção contra injeção de HTML

Todo valor vindo do banco passa por `escapeHtml()` antes de entrar na página. A função converte os caracteres que o navegador interpretaria como marcação:

```
function escapeHtml(valor) {
  if (valor === null || valor === undefined) return "";
  return String(valor)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}
```

Sem esse cuidado, um nome de asteroide contendo `<script>` seria executado pelo navegador de quem abrisse o painel. Os dados aqui vêm da NASA e são confiáveis, mas a prática vale para qualquer origem externa — o hábito é o que protege quando a fonte muda.

9 Validar a implementação

Sete verificações que confirmam o funcionamento de ponta a ponta.

	Verificação	Onde conferir
☐	RLS ativo nas tabelas	SQL Editor: consulta a `pg_tables` retorna `rls_ativo = true` nas duas
☐	Constraint UNIQUE criada	SQL Editor: `asteroides_unique` com tipo `u`
☐	Workflow verde	Aba Actions, execução com ícone verde
☐	Dados na tabela principal	Table Editor → `asteroides` com linhas
☐	Log registrado	Table Editor → `execucoes` com status `concluido`
☐	API REST respondendo	URL no navegador retorna JSON
☐	Painel carregando	Régua desenhada, tabela preenchida, filtro funcional

9.1 Consultas de verificação

```
-- RLS ativo nas duas tabelas
SELECT tablename, rowsecurity AS rls_ativo
FROM pg_tables
WHERE schemaname = 'public'
  AND tablename IN ('asteroides','execucoes');

-- Constraint UNIQUE presente
SELECT conname, contype FROM pg_constraint
WHERE conrelid = 'asteroides'::regclass;
```

> **Resultado parcial no SQL Editor.** Quando o script termina com mais de uma consulta, o Supabase exibe apenas o resultado da última. Para conferir as duas, rode-as separadamente.

9.2 Testar a API REST no navegador

```
https://SEU-PROJETO.supabase.co/rest/v1/asteroides
    ?select=*&order=distancia_lunar.asc&limit=10
    &apikey=SUA_PUBLISHABLE_KEY
```

Deve retornar um array JSON com as aproximações mais próximas. Um array vazio (`[]`) indica que o pipeline ainda não rodou. Um objeto de erro indica problema de permissão — revise RLS e GRANTs.

9.3 Problemas frequentes

Sintoma	Causa provável
Workflow não aparece no Actions	O arquivo `.github/workflows/update-data.yml` não foi criado — subir por upload não funciona
Erro de arquivo não encontrado	Arquivos enviados sem a estrutura de pastas
`HTTP 429`	Limite da NASA atingido. Aguarde a renovação horária
`erro 21000`	Deduplicação ausente ou desligada
Painel vazio, sem erro	Pipeline não rodou, ou URL/chave incorretas no `index.html`
Painel com erro de permissão	GRANT de `SELECT` ausente para `anon`

10 Adaptar para outra API

A arquitetura é a mesma para qualquer fonte de dados. Muda o que se coleta, não como se coleta.

Para replicar este projeto com outra API, quatro decisões precisam ser refeitas. O resto — workflow, estrutura de permissões, formato do painel — permanece igual.

Decisão	Como definir
Chave natural	Quais campos, juntos, identificam um registro de forma única e estável. É a constraint `UNIQUE` e o `on_conflict` do upsert. Definir errado aqui gera duplicata ou sobrescrita indevida.
Formato da resposta	Lista plana, objeto agrupado, paginação. Determina como o script achata os dados.
Colunas	Quais campos da API têm valor analítico. Nem tudo que vem precisa ser gravado.
Autenticação	Chave em query string, header, ou nenhuma. Se houver chave, ela é Secret.

> **Teste o endpoint no navegador antes de escrever o script.** Colar a URL com a chave e observar o JSON retornado revela o formato real da resposta em segundos — e evita descobrir uma estrutura inesperada no meio da depuração.

Referências

- Portal de APIs da NASA — `api.nasa.gov`
- Documentação do Supabase — `supabase.com/docs`
- Row Level Security no PostgreSQL — `postgresql.org/docs/current/ddl-rowsecurity.html`
- Sintaxe de agendamento do GitHub Actions — `docs.github.com/actions`